长广溪智能制造（无锡）有限公司
Changguangxi Intelligent Manufacturing (Wuxi) Co., Ltd.

# 协作机器人 ROS2 使用手册

文档版本：V1.1
发布日期：2025 年 03 月 04 日
文档编号：CGXi-QM-CR-012



**长广溪智能制造（无锡）有限公司**
**Changguangxi Intelligent Manufacturing(Wuxi) Co., Ltd.**



长广溪智能制造（无锡）有限公司（简称长广溪智造）保留对此文件修改之权利且不另行通知。长广溪智能制造（无锡）有限公司所提供之信息相信为正确且可靠之信息，但并不保证本文件中绝无错误。请于向长广溪智能制造（无锡）有限公司提出订单前，自行确定所使用之相关技术文件及规格为最新之版本。若因贵公司使用本公司之文件或产品，而涉及第三人之专利或著作权等智能财产权之应用及配合时，则应由贵公司负责取得同意及授权，本公司仅单纯贩售产品，上述关于同意及授权，非属本公司应为保证之责任。又未经长广溪智能制造（无锡）有限公司之正式书面许可，本公司之所有产品不得使用于医疗器材，维持生命系统及飞航等相关设备。

Copyright © 2022-2023 长广溪智造保留所有版权

# 目录

一、ROS2 简介 ..... 1
二、ROS2 控制 CGXI 机器人 ..... 3
2.1 准备工作 ..... 3
2.1.1 安装 ROS2 ..... 3
2.1.2 重要注意事项 ..... 4
2.2 控制 CGXI 机器人 ..... 5
2.2.1 设置终端环境 ..... 5
2.2.2 CGXI_ROS 驱动接口 ..... 6
2.2.3 Moveit 和 Gazebo 联合仿真 ..... 12
2.2.4 Moveit 和真实机器人联合使用 ..... 15

# 一、ROS2 简介

ROS2（Robot Operating System 2）为开源的机器人元操作系统的第二代版本，是一个专为机器人开发设计的软件。ROS2 继承了 ROS1 的核心思想，并在其基础上进行了大量改进和优化，以更好地满足现代机器人系统的需求。

ROS2 集成了丰富的工具、库和通信协议，提供了类似操作系统的基本功能，能够实现对机器人的高效控制和管理。

ROS2 常见的功能包结构如图 1-1 所示。

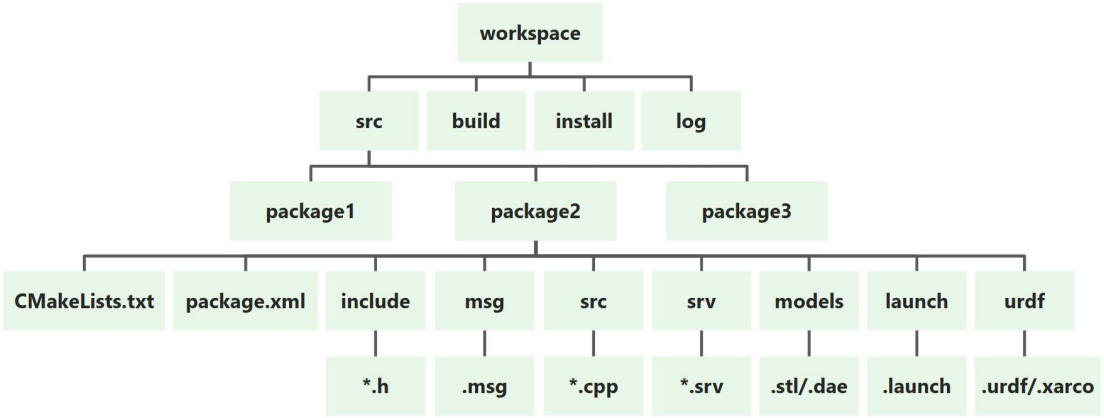



图 1-1 ROS2 功能包结构

CGXI 机器人的 ROS2 功能包结构大致如图 1-2 所示：

**注意：**工作空间（cgx_ros）中的 build 文件夹、install 文件夹和 log 文件夹，需要重新编译才能生成。

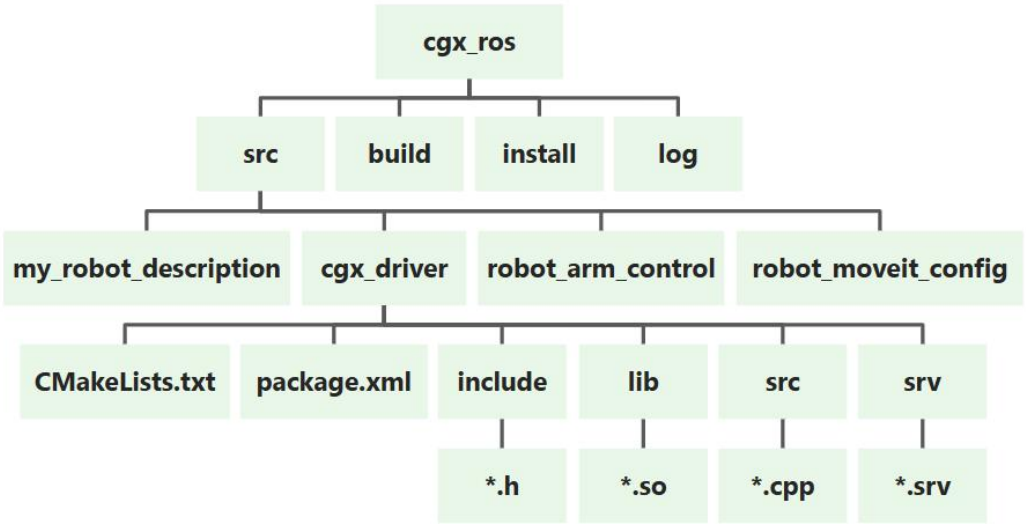



图 1-2 cgx_ros 功能包结构

各功能包说明如下：

| 名称 | 含义 |
| --- | --- |
| my_robot_description | 机器人模型描述文件 |
| cgx_driver | 真实机器人的启动控制功能包 |
| robot_arm_control | Moveit2 驱动真实机器人功能包 |
| robot_moveit_config | 机器人 Moveit2 配置文件 |

# 二、ROS2 控制 CGXI 机器人

**软件要求：Ubuntu2022.04、ROS2-humble**

## 2.1 准备工作

### 2.1.1 安装 ROS2

以下是在 Ubuntu 22.04 上安装 ROS2 Humble 的步骤：

（1）设置 Ubuntu 软件仓库

打开终端，运行以下命令以添加 ROS2 软件仓库到系统中：

```
sudo apt update
sudo apt install curl gnupg lsb-release
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o
/usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg]
http://packages.ros.org/ros2/ubuntu $(source /etc/os-release && echo $UBUNTU_CODENAME) main" |
sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

（2）更新软件包索引

添加仓库后，更新软件包索引：

```
sudo apt update
```

（3）安装 ROS2

根据需要选择安装不同的 ROS2 版本。以下是安装 ROS2 Humble 桌面完整版（包括 ROS2、RViz2、演示工具等）的命令：

```
sudo apt install ros-humble-desktop
```

（4）初始化 ROS2 环境

安装完成后，需要初始化 ROS2 环境。在终端运行以下命令：

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

（5）安装 colcon 构建工具

colcon 是 ROS2 推荐的构建工具。安装命令如下：

```
sudo apt install python3-colcon-common-extensions
```

（6）安装 rosdep

rosdep 用于管理 ROS2 软件包的依赖项。安装并初始化 rosdep：

```
sudo apt install python3-rosdep
sudo rosdep init
rosdep update
```

（7）创建工作空间

创建一个 ROS2 工作空间来组织和构建自己的 ROS2 软件包。将 cgx_ros 包放至跟目录下：

```
cd ~/cgx_ros
colcon build
```

构建完成后，可以通过以下命令加载工作空间环境：

```
source ~/cgx_ros/install/setup.bash
```

（8）开始使用 ROS2

现在你已经成功安装了 ROS2 Humble，并创建了一个工作空间。可以开始编写、构建和运行 ROS2 软件包了。

以上步骤适用于 Ubuntu 22.04 和 ROS2 Humble。如果使用其他版本的 Ubuntu 或 ROS2，请替换相应的版本代号。更多详细信息和指导，请参考 ROS2 官方文档：https://docs.ros.org/en/humble/。

### 2.1.2 重要注意事项

（1）cgx_ros 包目前只支持 ROS2（Humble）版本，不支持 ROS1 版本。请确保在 ROS2 环境下进行开发和测试。

（2）cgx_ros 包目前仅支持 x86_64 架构的 Ubuntu 系统。其他架构（如 ARM）暂不支持。

（3）手册中的案例是在 ROS2 Humble 版本下测试的。如果在使用过程中出现报错，建议执行以下操作：删除工作空间（如 cgx_ros）中的 build、install 和 log 文件夹。重新使用 colcon build 命令编译工作空间。

（4）cgx_ros 包通过 CGXI SDK 提供的函数接口控制机器人。相关接口的具体说明可以参考《CGXi-协作机器人-sdk 使用手册（C_C++）》。请确保已正确安装并配置 CGXI SDK。

（5）在运行 cgx_ros 时，cgx_driver 服务端和 MoveIt 服务端不能同时启动，否则可能会导致资源冲突或控制异常。请根据实际需求选择启动其中一个服务端。

（6）在使用 CGXI_ROS 驱动接口 cgx_driver 服务端以及 Moveit 和真实机器人联合使用之前，请确保已正确配置 CGXI 的 C++ SDK 环境。可以通过以下命令配置 CGXI 的 C++ SDK 环境变量：

```
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:~/cgx_ros/src/cgx_driver/lib
```

（7）在使用 cgx_ros 包之前，请确保已正确配置 ROS2 环境变量。可以通过以下命令加载环境变量：

```
source /opt/ros/humble/setup.bash
source ~/cgx_ros/install/setup.bash
```

## 2.2 控制 CGXI 机器人

### 2.2.1 设置终端环境

在 ROS2 中，每次打开一个新的终端时，需要运行 source install/setup.bash 来设置正确的环境变量。为了避免每次手动执行这个命令，可以按照以下的方法修改.bashrc 文件，使得每次打开终端时自动加载 ROS2 工作空间。

（1）修改.bashrc 文件

在.bashrc 文件末尾添加一行，用于自动加载 ROS2 工作空间。假设你的 ROS2 工作空间位于 ~/cgx_ros 目录下，通过以下命令将 source ~/cgx_ros/install/setup.bash 添加到.bashrc 文件中：

```
echo "source ~/cgx_ros/install/setup.bash" >> ~/.bashrc
```

（2）使配置生效

执行以下命令使修改后的.bashrc 文件立即生效：

```
source ~/.bashrc
```

（3）确认是否写入成功

可以通过以下命令打开.bashrc 文件，检查是否成功添加了 source ~/cgx_ros/install/setup.bash 这一行：

```
gedit ~/.bashrc
```

在打开的文件中，你应该能看到类似如下的最后一行：

```
source ~/cgx_ros/install/setup.bash
```

（4）验证设置

关闭当前终端窗口并重新打开一个新的终端窗口，然后运行以下命令来验证环境变量是否已正确设置：

```
echo $ROS_DISTRO
```

如果输出显示了你当前使用的 ROS2 发行版（humble），则说明环境变量已正确加载。

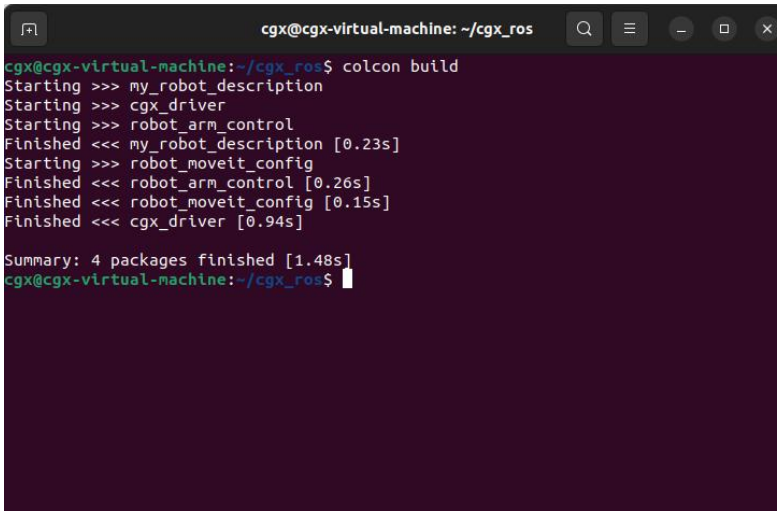



图 2-1 cgx_ros 构建示意图

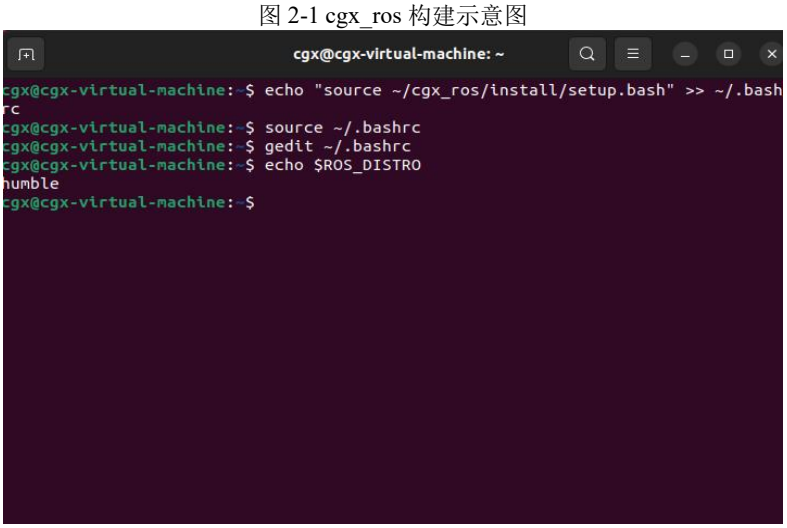



图 2-2 添加环境变量示意图

**注意：**如果没有进行进行该设置，下面的操作每次打开一个终端都需要通过“source install/setup.bash”命令来添加环境变量。

## 2.2.2 CGXI_ROS 驱动接口

实现功能：启动 CGXI_ROS 驱动服务端，通过 ros2 service call 调用带参数的各功能服务。定义服务接口的文件为~/cgx_ros/src/cgx_driver/CgxDriverService.srv 文件，.srv 文件用于定义服务的请求（request）和响应（response）数据结构。其数据结构如下所示。详细信息可参考《CGXi-协作机器人-sdk 使用手册（C_C++）》。

| 类别 | 数据格式 | 定义 |
| --- | --- | --- |
| request | float32[] pose | 坐标点向量：x，y，z（单位：mm）；Rx， |

| | | |
|---|---|---|
| | | Ry，Rz（单位：°） |
| | float32[] jointpos | 关节角度：关节 1~6（单位：° ） |
| | float32[] speed | 运动速度 |
| | float32[] acc | 运动加速度 |
| | string coordinate_type | 坐标系类型 |
| | string point_trans_type | 位置过渡方式 |
| | float32 point_trans_radius | 过渡半径 |
| | int16 is_block | 阻塞类型 |
| response | int16 ret | 返回值，表示成功或失败 |
| | string message | 返回值，表示调用结果信息 |

调用机器人各功能服务的操作步骤如下：

（1）在工作空间（cgx_ros）右键打开一个终端，输入以下命令配置 CGXI 的 C++ SDK 环境。

```
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:~/cgx_ros/src/cgx_driver/lib
```

（2）使用以下命令启动 cgx_driver 服务端。

```
ros2 run cgx_driver cgx_driver_node
```

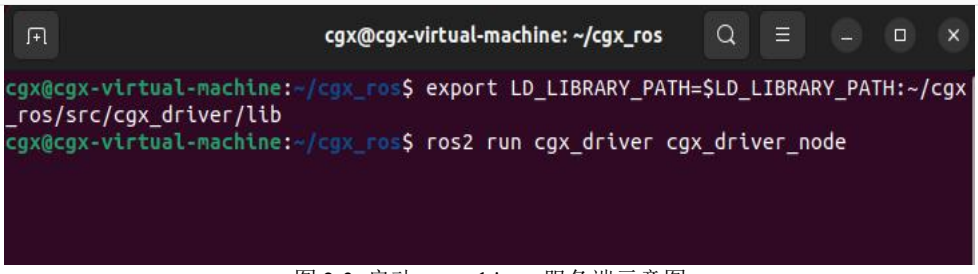



图 2-3 启动 cgx_driver 服务端示意图

（3）轴空间运动 cr_movej：使用 ros2 service call /cr_moveJ cgx_driver/srv/CgxDriverService 并输入对应的参数，控制机器人运动。案例将运动接口的坐标系类型设置为关节坐标系，将位置过渡方式设置为过渡点停止，过渡半径设置为 0，可以通过修改对应参数的方式实现修改运动接口。

```
ros2 service call /cr_moveJ cgx_driver/srv/CgxDriverService "{
pose: [0, 0, 0, 0, 0, 0],
jointpos: [60, 60, -60, 60, 60, 60],
speed: [30, 30, 30, 30, 30, 30],
acc: [30, 30, 30, 30, 30, 30],
coordinate_type: 'joint',
point_trans_type: 'stop',
point_trans_radius: 0,
is_block: 0
}"
```

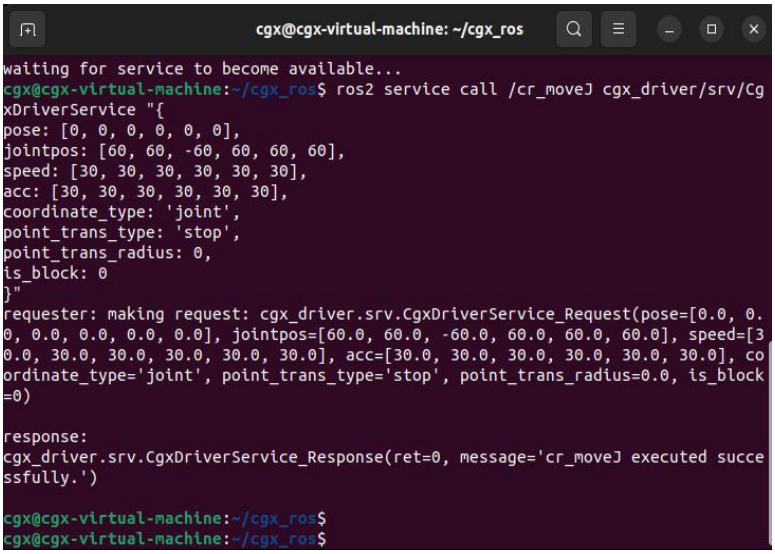



图 2-4 ros2 调用 cr_moveJ 服务示意图




图 2-5 机器人示意图

（4）轴空间运动 cr_move_joint：使用 ros2 service call /cr_move_joint cgx_driver/srv/CgxDriverService 并输入对应的参数，控制机器人运动。案例将运动接口的坐标系类型设置为关节坐标系，将位置过渡方式设置为匀速过渡，过渡半径设置为 5，阻塞类型设置为阻塞模式。可以通过修改对应参数的方式实现修改运动接口。

```
ros2 service call /cr_move_joint cgx_driver/srv/CgxDriverService "{
pose: [0, 0, 0, 0, 0, 0],
jointpos: [-60, -60, 60, -60, -60, -60],
speed: [30, 30, 30, 30, 30, 30],
acc: [30, 30, 30, 30, 30, 30],
coordinate_type: 'joint',
point_trans_type: 'speed',
```

point_trans_radius: 5,

is_block: 1

}"

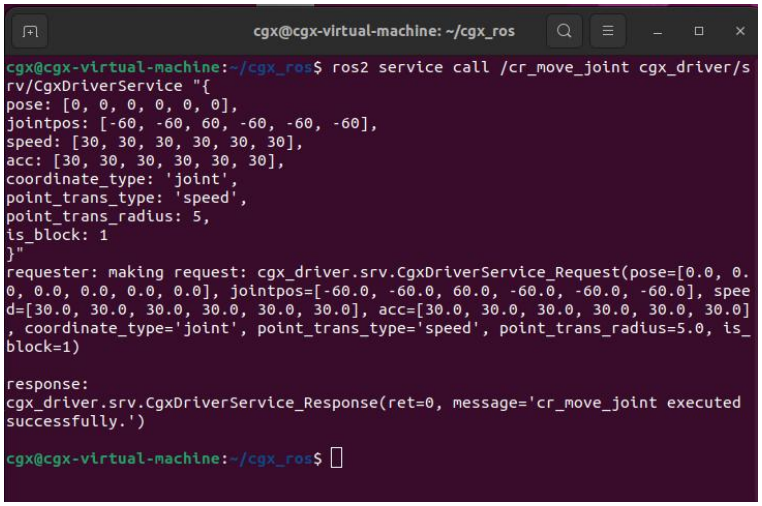



图 2-6 ros2 调用 cr_move_joint 服务示意图

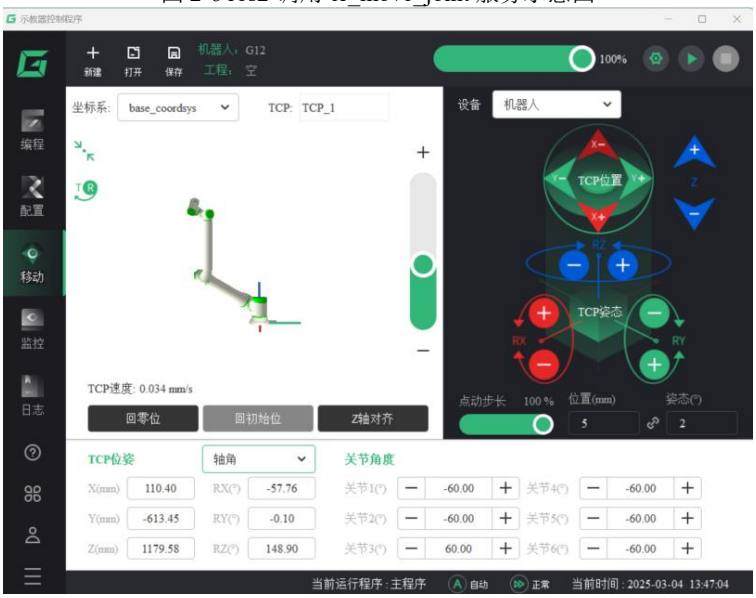



图 2-7 机器人示意图

（5）直线运动 cr_moveL：使用 ros2 service call /cr_moveL cgx_driver/srv/CgxDriverService 并输入对应的参数，控制机器人运动。案例将运动接口的坐标系类型设置为基坐标系，将位置过渡方式设置为过渡点停止，过渡半径设置为 0。可以通过修改对应参数的方式实现修改运动接口。

ros2 service call /cr_moveL cgx_driver/srv/CgxDriverService "{

pose: [3, -588, 1266, -57.34, 1, 149],

jointpos: [-60, -60, 60, -60, -60, -60],

speed: [30, 30, 30, 30, 30, 30],

acc: [30, 30, 30, 30, 30, 30],

coordinate_type: 'base',

```
point_trans_type: 'stop',
point_trans_radius: 0,
is_block: 0
}"
```

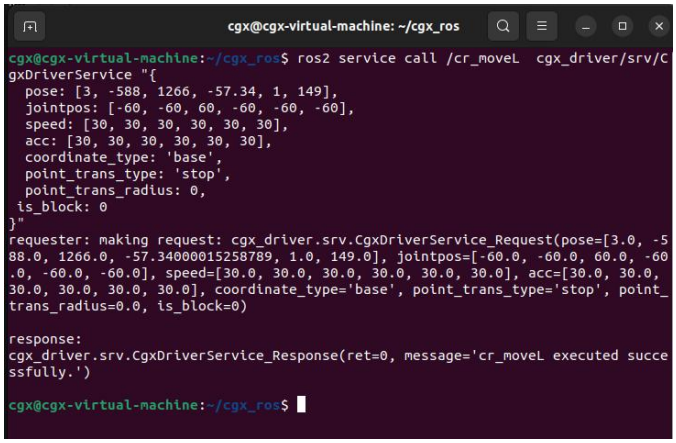



图 2-8 ros2 调用 cr_moveL 服务示意图

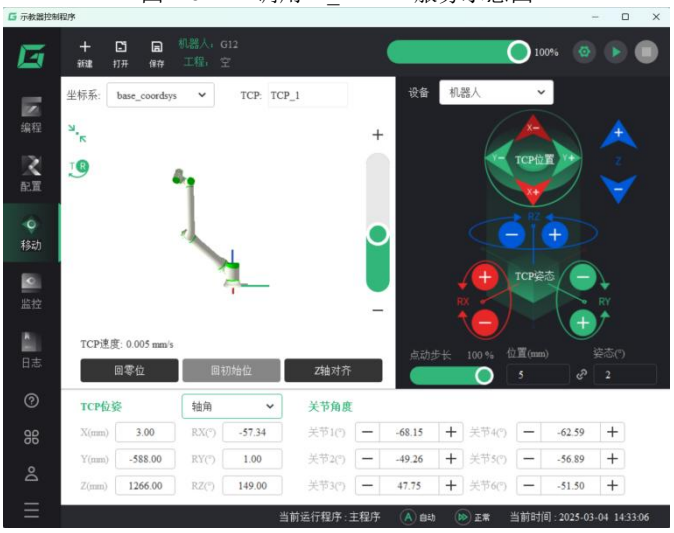



图 2-9 机器人示意图

（6）直线运动 cr_move_line：使用 ros2 service call /cr_move_line cgx_driver/srv/CgxDriverService 并输入对应的参数，控制机器人运动。案例将运动接口的坐标系类型设置为基坐标系，将位置过渡方式设置为匀速过渡，过渡半径设置为 5，阻塞类型设置为非阻塞模式。可以通过修改对应参数的方式实现修改运动接口。

```
ros2 service call /cr_move_line cgx_driver/srv/CgxDriverService "{
pose: [6, -600, 1300, -57.34, 1, 149],
jointpos: [-60, -60, 60, -60, -60, -60],
speed: [30, 30, 30, 30, 30, 30],
acc: [30, 30, 30, 30, 30, 30],
coordinate_type: 'base',
```

```
point_trans_type: 'speed',
point_trans_radius: 5,
is_block: 0
}"
```

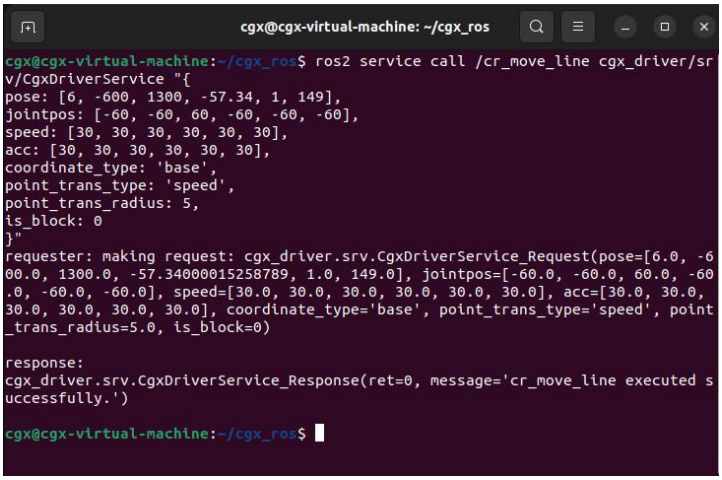



图 2-10 ros2 调用 cr_move_line 服务示意图

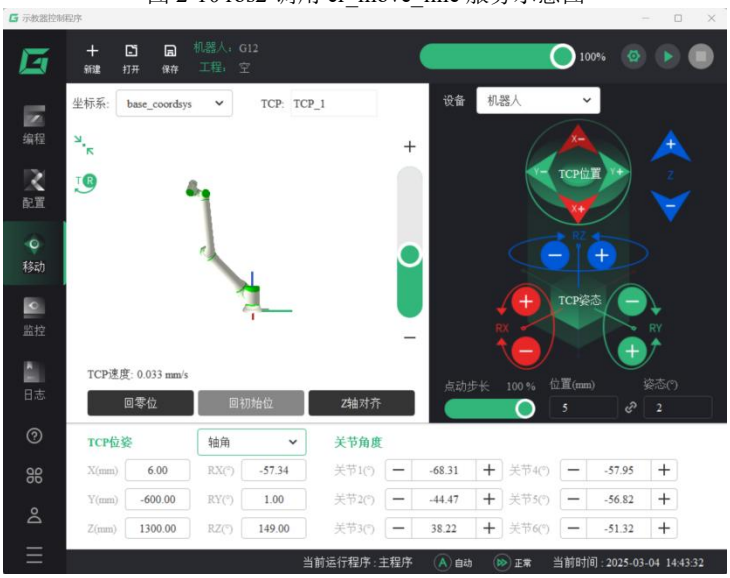



图 2-11 机器人示意图

（7）点动运动 cr_moveJog：使用 ros2 service call /cr_moveJog cgx_driver/srv/CgxDriverService 并输入对应的参数，控制机器人运动。案例将运动接口的坐标系类型设置为基坐标系，将位置过渡方式设置为匀速过渡，过渡半径设置为 5，阻塞类型设置为非阻塞模式。可以通过修改对应参数的方式实现修改运动接口。

```
ros2 service call /cr_moveJog    cgx_driver/srv/CgxDriverService "{
pose: [0, 5, 0, 0, 0, 0],
jointpos: [60, 60, -60, 60, 60, 60],
speed: [30, 30, 30, 30, 30, 30],
```

```
acc: [30, 30, 30, 30, 30, 30],
coordinate_type: 'base',
point_trans_type: 'stop',
point_trans_radius: 0,
is_block: 1
}"
```

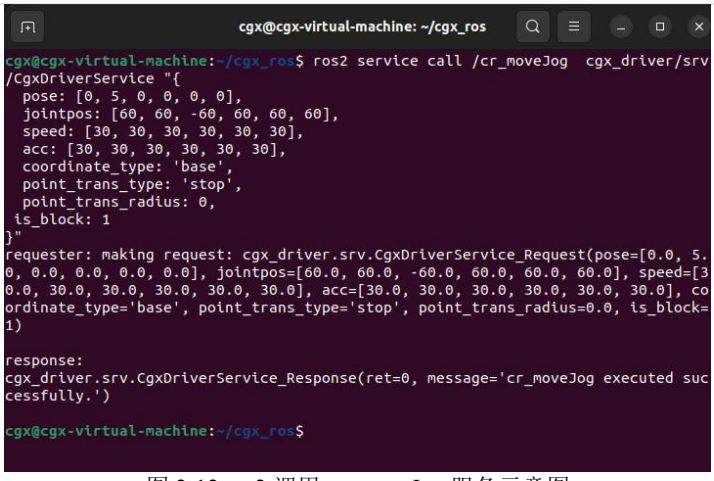



图 2-12 ros2 调用 cr_moveJog 服务示意图




图 2-13 机器人示意图

## 2.2.3 Moveit 和 Gazebo 联合仿真

实现功能：启动 Rviz 和 Gazebo，在 Rviz 中规划路径并执行，在 Gazebo 中的机器人仿真模型会移动到相应位置。

（1）需要先启动 Gazebo，在工作空间（cgx_ros）右键打开一个终端，使用以下命令，可以在 Gazebo 中看到 G6 仿真机器人模型。

```
ros2 launch my_robot_description g6_launch.py
```

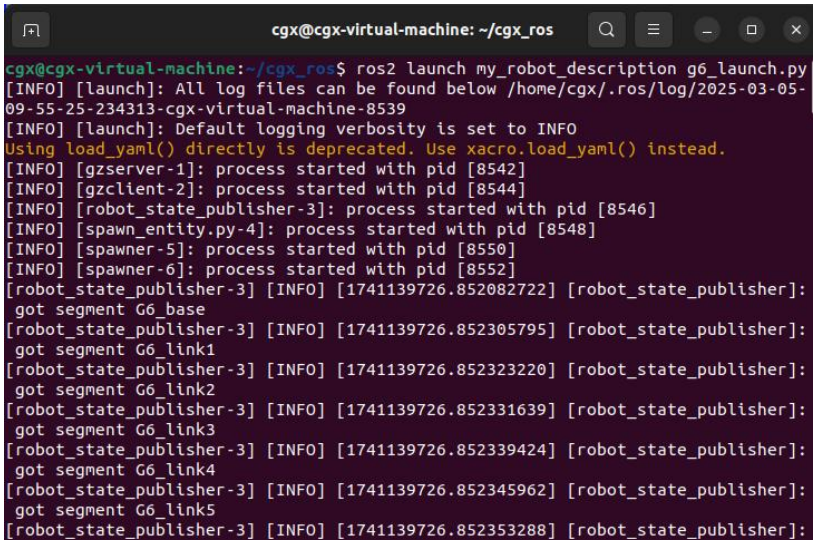



图 2-14 启动 Gazebo 示意图

（2）在工作空间（cgx_ros）右键打开一个新的终端，使用以下命令，启动 Rviz。

```
ros2 launch robot_moveit_config my_moveit_rviz.launch.py
```

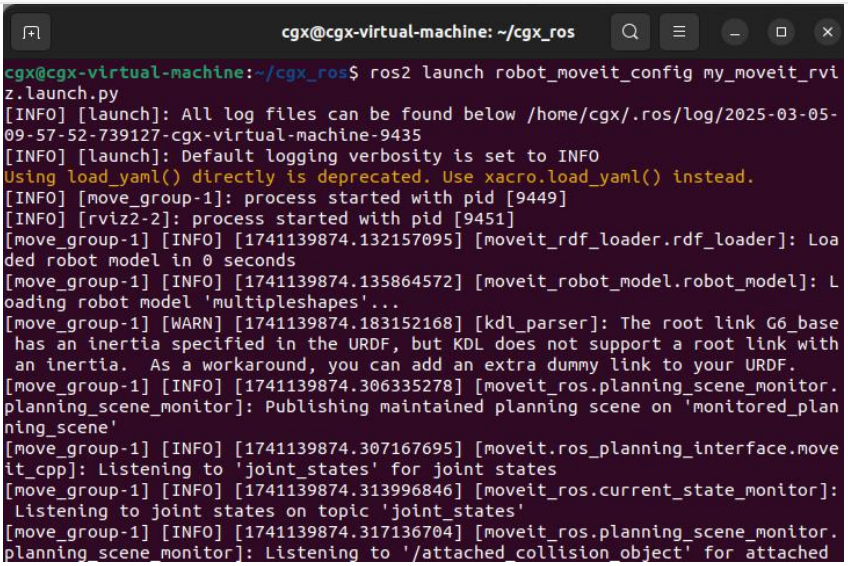



图 2-15 启动 Rviz 示意图

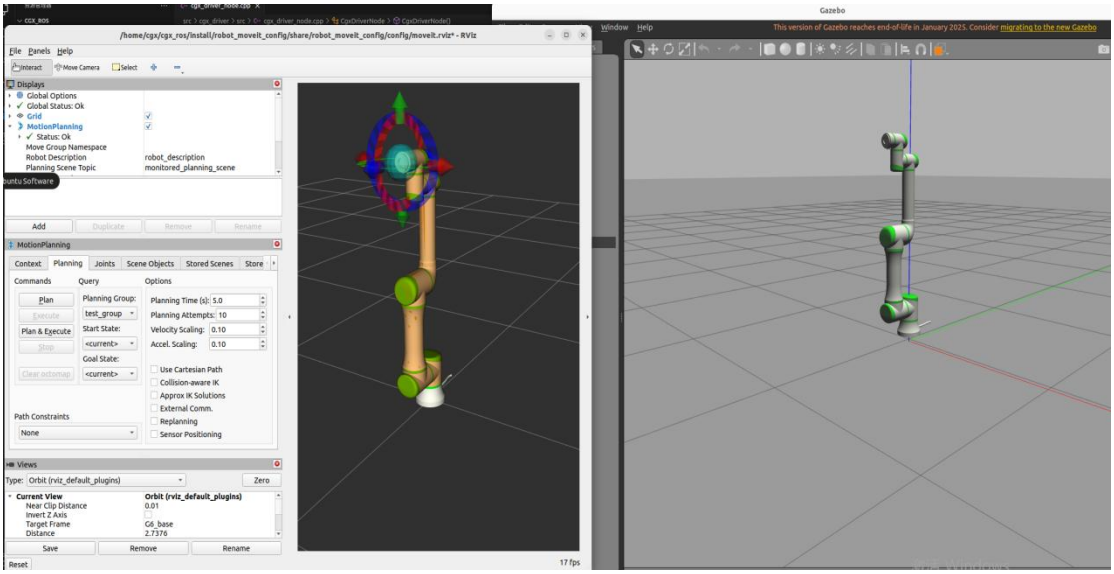



图 2-16 Rviz 和 Gazebo 机器人示意图

（3）从 Rviz 界面的“Goal state”选择一个目标位置，或者在 Joints 修改关节数据，点击“Plan & Execute”，会显示机器人的轨迹，并驱动 Gazebo 中仿真机器人到设置的目标位置。

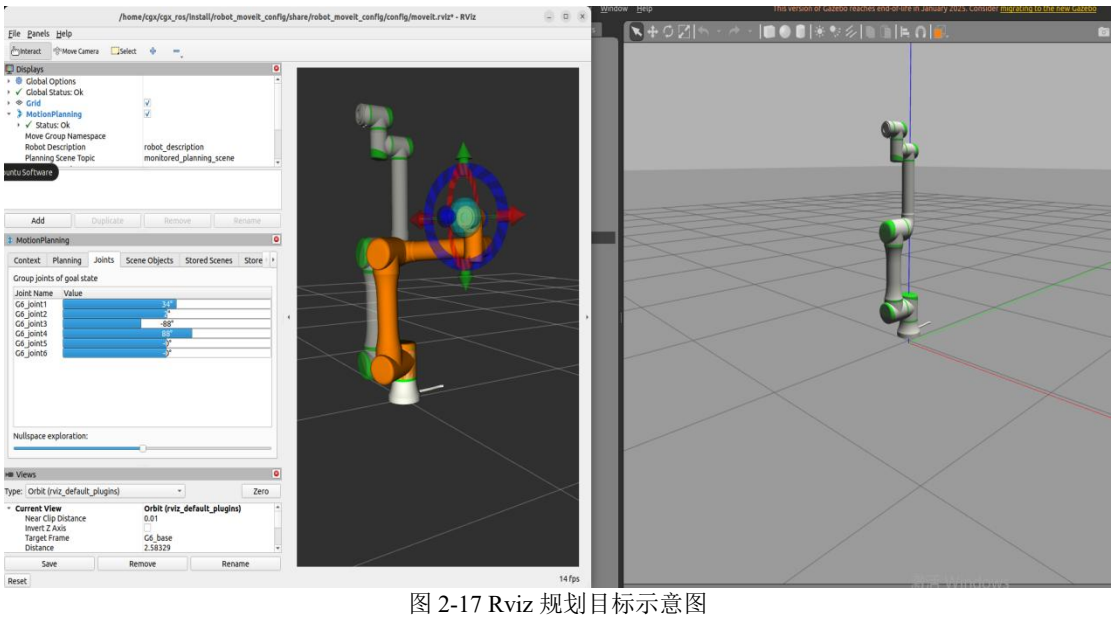



图 2-17 Rviz 规划目标示意图

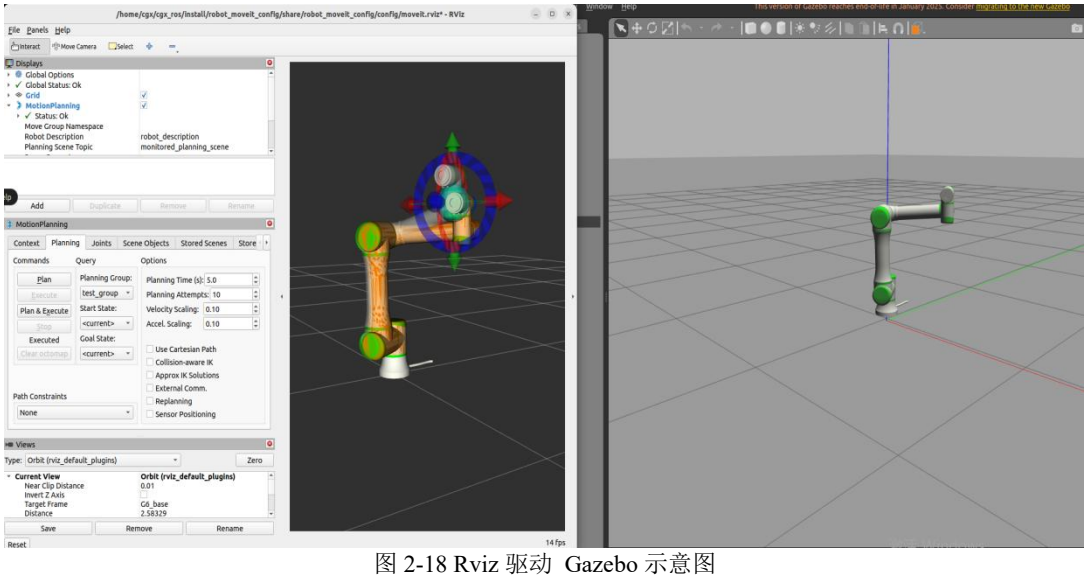


图 2-18 Rviz 驱动 Gazebo 示意图

## 2.2.4 Moveit 和真实机器人联合使用

实现功能：启动 moveit 服务端及 RVIZ，RVIZ 规划路径，真实机械臂会移动到相应位置。

注意事项：cgx_driver 服务端和 Moveit 服务端不能同时启动，所以启动 moveit 服务端之前确认 cgx_driver 服务端已经关闭（即未启动 ros2 run cgx_driver cgx_driver_node）。

（1）在工作空间（cgx_ros）右键打开一个终端，输入以下命令配置 CGXI 的 C++ SDK 环境。

```
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:~/cgx_ros/src/cgx_driver/lib
```

（2）需要先启动 moveit_controller，使用以下命令，可以启动 moveit_controller 服务端。

```
ros2 run robot_arm_control moveit_controller
```

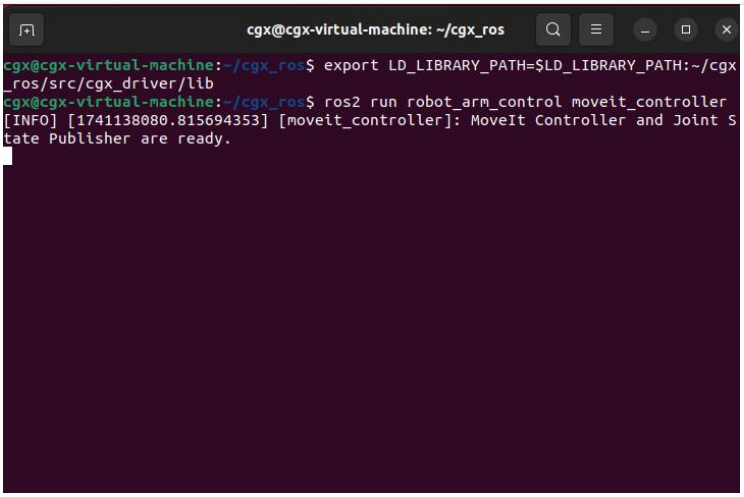



图 2-19 启动 moveit_controller 示意图

（3）在工作空间（cgx_ros）右键打开一个新的终端，使用以下命令，启动 moveit 客户端。

```
ros2 launch robot_moveit_config demo.launch.py
```

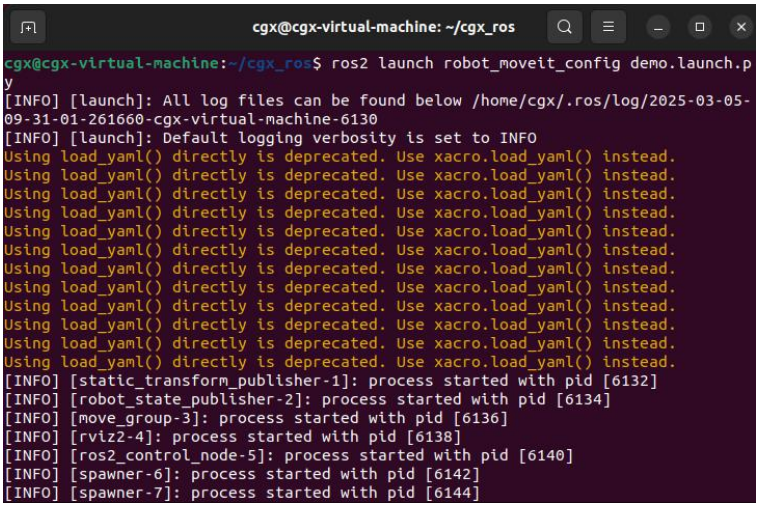



图 2-20 启动 moveit 客户端示意图

（4）可以看到 Rviz 界面显示的机器人模型姿态和实体机器人当前姿态一致。从 Rviz 界面的“Goal state”选择一个目标位置，或者在 Joints 修改关节数据，点击“Plan & Execute”，会显示机器人的轨迹，并驱动真实机器人到设置的目标位置。

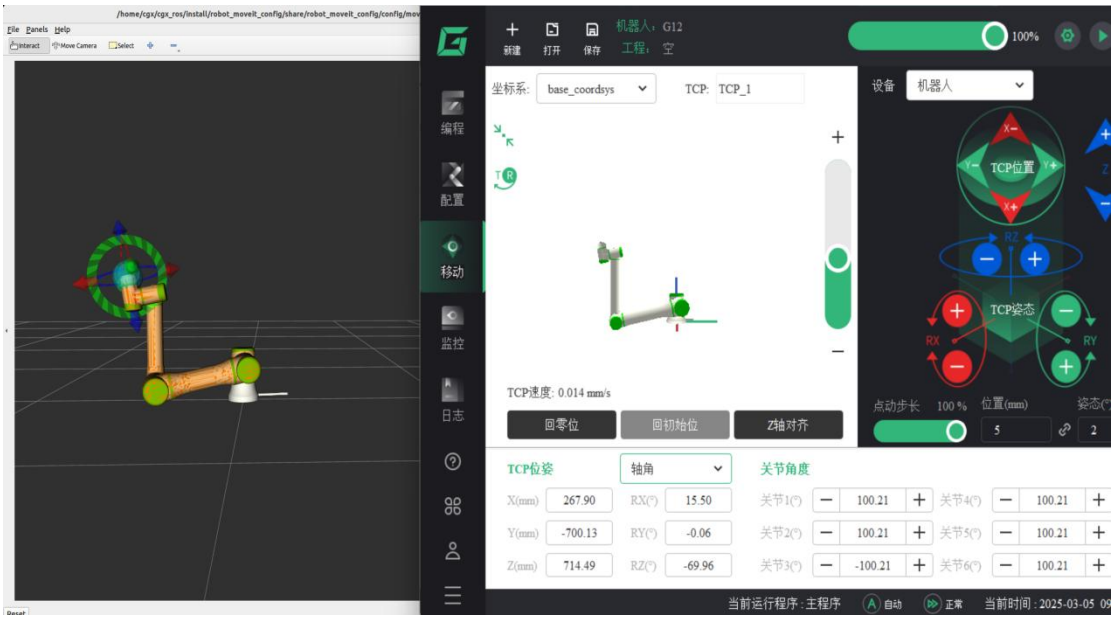



图 2-21 Rviz 和真实机器人示意图

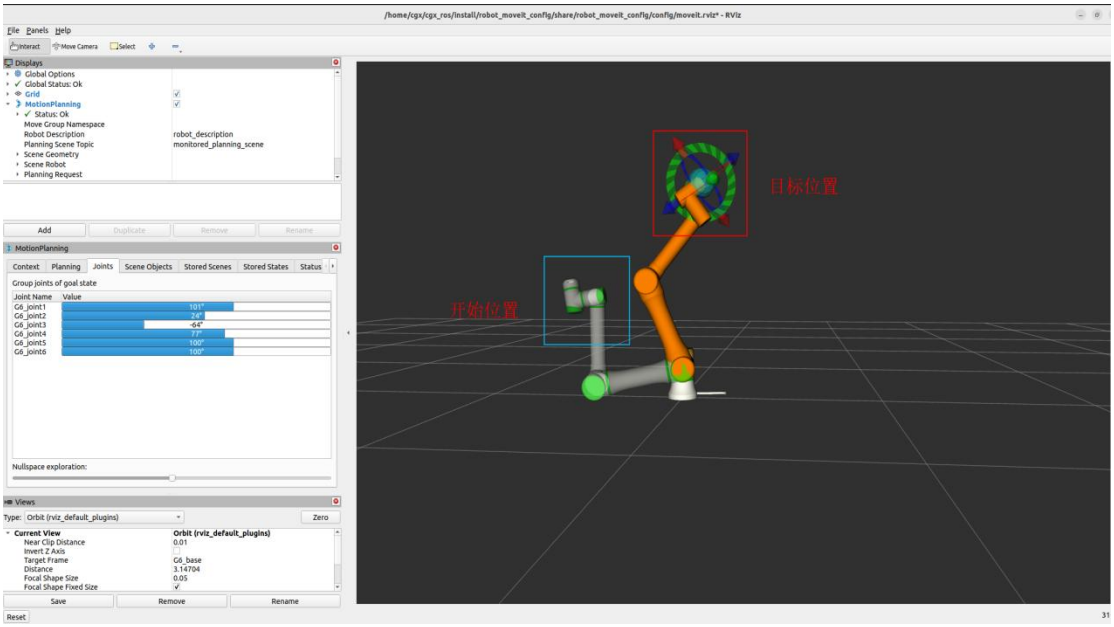



图 2-22 Rviz 目标规划示意图

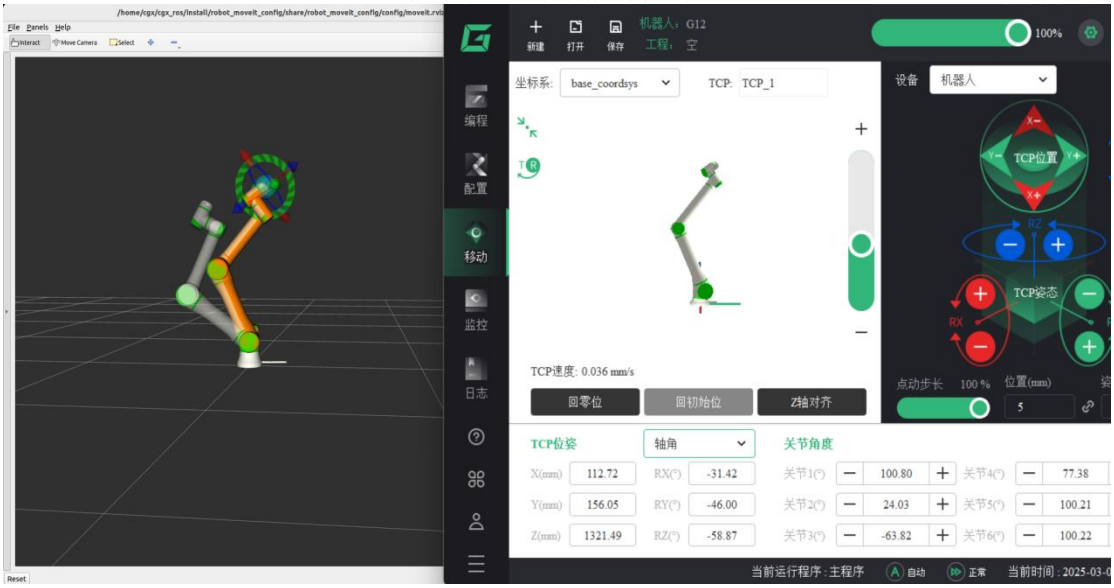



图 2-23 Rviz 驱动真实机器人示意图